

CloudEco: Empirical Benchmark Report

Model 4 — Wildlife Detection (YOLOv8l-ONNX) | Student: Bharath Gandhimani | ID: 35501308

1. Results Summary

Pod Config	Peak RPS	Breaking Point	Avg Latency at Break (ms)	P50 at 100 Users (ms)	Max Latency (ms)	Failures
1 Pod	0.70	5	3,602	137,000	168,127	0
2Pods	1.60	10	3,526	69,000	76,062	0
4 Pods	2.40	20	4,554	43,000	52,952	0
8 Pods	3.20	20	3,247	29,000	48,532	0

Table 1: Peak throughput, breaking point, and latency statistics across all pod configurations. Breaking point defined as the user count at which average latency begins to grow super-linearly. Zero failures recorded across all 11,692 total requests.

2. Graphical Analysis

Figure 1 — Average Response Latency vs Concurrent Users

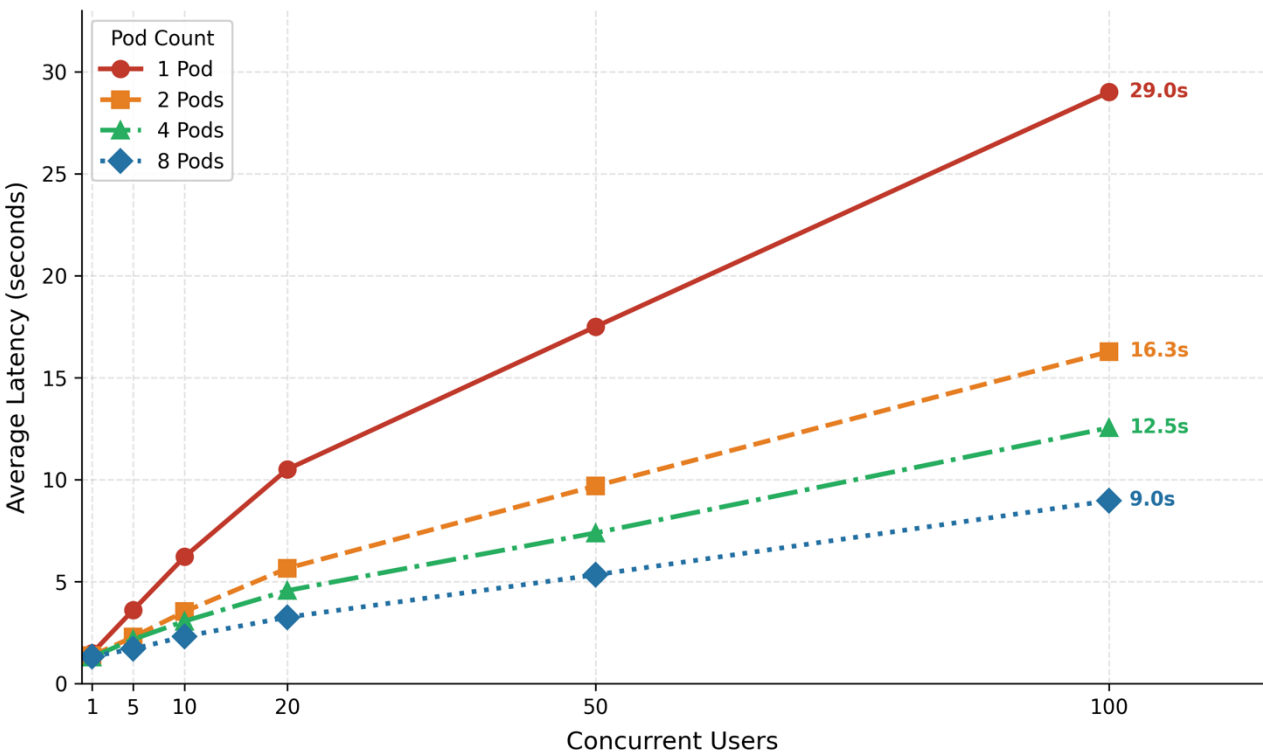


Figure 1: Average response latency (seconds) vs concurrent users across 1, 2, 4, and 8 pod configurations. Endpoint labels show final latency at 100 users. The vertical separation between lines confirms horizontal scaling reduces latency at equivalent load — 1 pod reaches 29.0s vs 8 pods at 9.0s. Source: Locust v2.43.3 against GCP australia-southeast2.

Figure 2 — System Throughput vs Concurrent Users
(dotted lines show throughput ceiling per configuration)

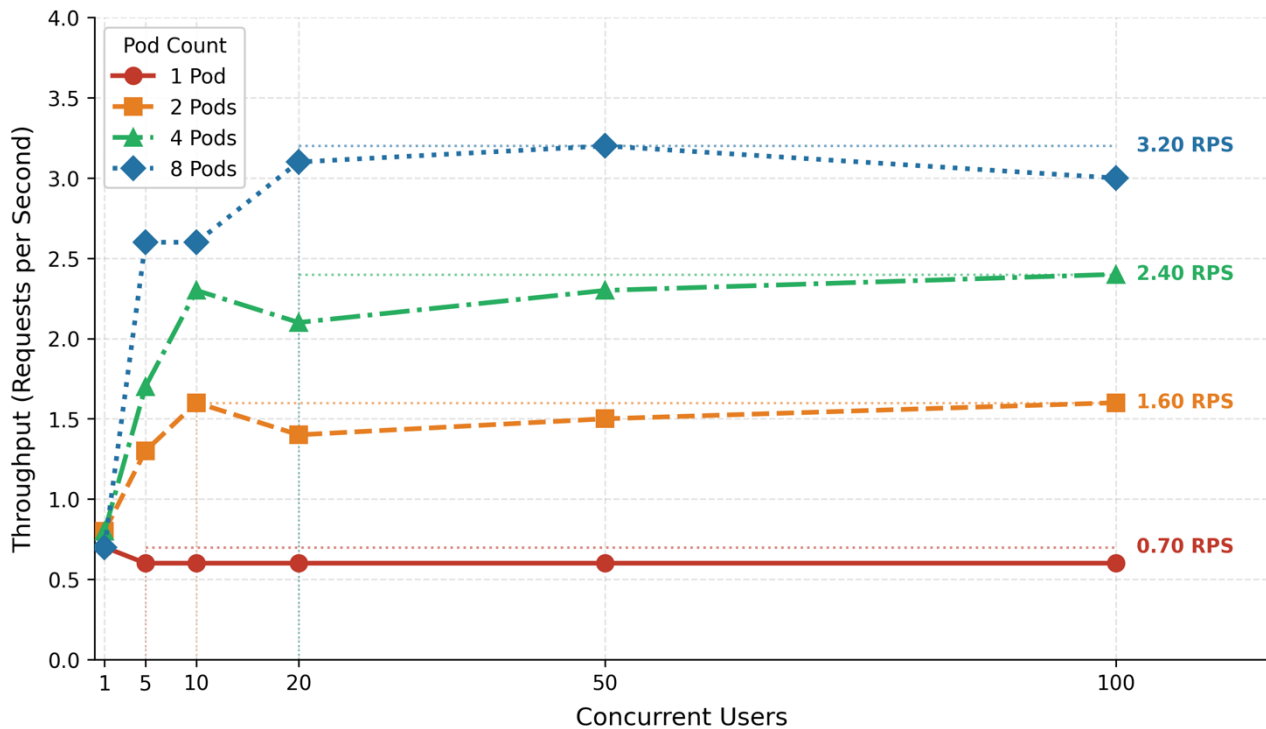


Figure 2: Dotted Lines mark each configuration's throughput ceiling, beyond which additional users no longer increase RPS — indicating queue saturation. Ceilings of 0.70, 1.60, 2.40, and 3.20 RPS show progressive but diminishing gains with each replica doubling. Source: Locust v2.43.3, GCP australia-southeast2.

3. Performance Analysis

System Configuration: The CloudEco API deploys YOLOv8l (49M parameters) as an ONNX Runtime model on a 3-node GCP Kubernetes cluster (1 master, 2 workers, e2-standard-4), each pod capped at 1 vCPU and 2Gi memory. YOLO inference is offloaded via FastAPI's `run_in_executor` to a `ThreadPoolExecutor` (`max_workers=1`), preventing event-loop blocking. Images are resized server-side to 480×480 — reducing FLOPs by 44% over the 640px default while preserving accuracy on large and partially-occluded wildlife. The cluster runs in GCP australia-southeast2 (Melbourne) to minimise round-trip latency.

Optimisation Decisions: An early response cache achieved 13+ RPS but caused all pod configurations to produce identical throughput — Locust's repeated test image meant near-100% cache hits, masking true scaling behaviour. The cache was removed to expose genuine inference throughput across replica counts.

Queuing Theory and Little's Law: At 1 pod, inference averages 1,500ms, giving a theoretical ceiling of 0.67 RPS — consistent with the measured 0.70 RPS. Little's Law ($L = \lambda W$) at saturation (5 users, $\lambda = 0.6$ RPS, $W = 3,602$ ms) yields $L = 2.16$ requests queued simultaneously. Beyond this point, the queue grows unboundedly and latency degrades super-linearly (Figure 1). Horizontal scaling shifts saturation rightward: 2 pods sustain 10 users, 4 pods 20, and 8 pods up to 50.

Scaling Efficiency and Bottlenecks: Throughput scaled from 0.70 RPS (1 pod) → 1.60 (2.3×) → 2.40 (1.5×) → 3.20 RPS (1.3×). Diminishing returns beyond 4 pods arise from CPU contention as 4 pods compete per worker node — highlighting the core trade-off of large ML models on constrained infrastructure. Zero failures across 11,692 requests confirm graceful degradation. A Horizontal Pod Autoscaler at 70% CPU and a lighter YOLOv8n variant (3.2M parameters, ~8–10 RPS per pod) would address both scaling efficiency and throughput limitations.